

DYNAMIC WEIGHT GRAFTING: LOCALIZING FINE-TUNED FACTUAL KNOWLEDGE IN TRANSFORMERS

Anonymous authors
Anonymous submission

ABSTRACT

When an LLM is updated with a newly introduced fact during finetuning (e.g., new movie releases, newly elected pope, etc.), where in the computation does that update become usable? Does the model attach relation information to entities as soon as they are processed, retrieve the information only when a prediction is imminent, or rely on multiple partially redundant routes? Existing localization approaches (e.g., activation patching) are limited for this question because they usually *replace* parts of the residual stream, which can erase upstream information that may itself be part of the retrieval process. To study this broader class of finetuned knowledge-retrieval mechanisms, we propose **dynamic weight grafting**, an analysis technique that selectively grafts *subsets of weights* from a finetuned model onto a pretrained model. Across controlled relation-completion settings, this technique reveals two separable pathways for retrieving finetuned relation information: 1) “enriching” the residual stream with relation information while processing the tokens that correspond to an entity (e.g., “Zendaya” in “Zendaya co-starred with Timothée Chalamet” and 2) “recalling” this information at the final token position before generating a target fact. Depending on the model and prompt format, relation completion can require both pathways or can be supported by either the “enrichment” or “recall” pathway alone. We localize the “recall” pathway to model components—finding that “recall” occurs through both task-specific attention mechanisms and an entity-specific extraction step in the feedforward networks of the final layers before prediction. By targeting components and parameters rather than only activations, dynamic weight grafting provides a way to analyze the *mechanisms* by which finetuned knowledge is retrieved during generation.

1 INTRODUCTION

When an LLM is finetuned so that it can answer a newly relevant query such as “Who’s the pope?” how is that behavioral update implemented inside the model? Large Language Models (LLMs) store and retrieve many relationships and associations acquired during pretraining (Petroni et al., 2019; Roberts et al., 2020), but finetuning raises a more general mechanistic question: how are newly supplied facts encoded in parameters, and which internal computations make those facts available during text generation?

When new relationship information is added to an LLM, it could be represented in several places and used at several times: stored directly with the entity (e.g., in embeddings), “enriched” into the entity representation in lower or middle layers, or recalled in *response* to the entity in higher layers close to next-token prediction. At a finer scale, this framing asks which components support newly learned relation information, and at which token positions those components contribute to retrieval.

Our goal is to develop tools for isolating which *mechanisms* make finetuned information usable in context: for instance, whether a feedforward network in a late layer recalls that Pope Leo XIV was recently elected, or whether attention routes information that was already assembled in earlier hidden states. Prior approaches to localizing relation knowledge often use variants of *activation patching* (Meng et al., 2022a) or ablations (Geva et al., 2023) to measure which components affect next-token prediction. These interventions are powerful but have a key limitation for scope-sensitive localization: by modifying or replacing activations, they can block access to computations that occurred before the intervention. For example, patching or ablating an activation at a layer and token position also

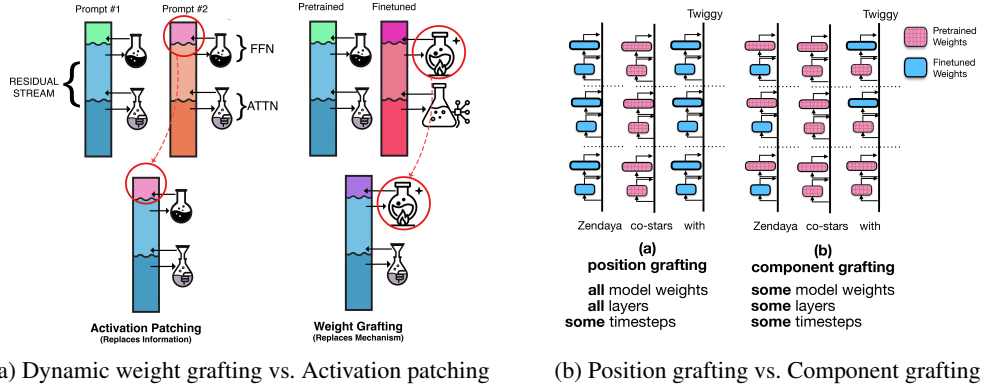


Figure 1: Dynamic weight grafting analyzes *model mechanisms* for finetuned knowledge retrieval by swapping *small subsets* of a pretrained model’s weights with the corresponding weights of a model that has undergone supervised finetuning (SFT). (a) **Comparing Dynamic Weight Grafting to Activation Patching:** Activation patching replaces model activations at a specific point with activations from another run, whereas dynamic weight grafting replaces *individual mechanisms* by inserting selected parameter matrices from a finetuned model into a pretrained model. (b) The grafting schemes used in our experiments. Position grafting uses either the entire pretrained or finetuned model at every layer and component for a given token position. Component grafting blends pretrained and finetuned weights dynamically at each token position.

overwrites upstream information flowing into that activation, including possible “enrichment” of an entity with associated factual information (Geva et al., 2023). This makes it difficult to distinguish a component that actively extracts new information from one that passes forward information computed earlier. Consequently, activation patching and ablations alone do not determine which parts of the model are both necessary *and* sufficient for recalling relation information, a standard we aim to support for finetuned knowledge localization.

To address this, we propose **dynamic weight grafting**,¹ a method for interpreting how finetuned knowledge is used in language models by swapping in subsets of weights from a finetuned model during generation. The swaps can be targeted to particular layers, components, and token positions (see Figure 1b), enabling tests of which parts of the model are necessary and sufficient to reproduce the effects of finetuning while leaving the surrounding computation available. This approach combines the sequence-preserving advantages of model grafting (Panigrahi et al., 2023; Ilharco et al., 2022) with the component-level causal questions typically pursued with activation patching (Heimersheim and Nanda, 2024; Goldowsky-Dill et al., 2023; Meng et al., 2022a).²

Using this method, we identify two pathways through which finetuned relation knowledge can be retrieved during generation. In some model-template combinations, grafting only at the first entity or only at the last token is sufficient to **reproduce the finetuned model’s** relation-completion behavior. This supports two complementary interpretations: (1) when an entity is “enriched” with relation information as it is processed, generic downstream mechanisms can extract the correct entity to complete the relation tuple (Meng et al., 2022a; Geva et al., 2023; 2022); and (2) “recall” mechanisms can also extract relation completions from representations that were not enriched with finetuned relation information. The two pathways together nearly recover full finetuning performance, whereas grafting everything *except* these pathways produces near-zero relation completion accuracy. Thus, within our tested relation-completion settings, dynamic weight grafting shows that enrichment and recall pathways are necessary *and* sufficient for the observed behavior and provides a finer-grained localization of finetuned information.

¹Code will be released upon acceptance.

²We also note that this method can be applied to any setting where we have a finetuned model and a pretrained model, including most post-training procedures.

2 METHODOLOGY

Given pretrained and finetuned model parameters θ^{pre} and θ^{ft} , we ask how to localize the mechanisms responsible for the finetuned model’s behavioral changes. A relation-completion behavior may be built gradually as each token is processed, or it may be retrieved by a later computation that “looks back” while processing the final token before prediction. Interpretability work has often addressed such questions by intervening on the *information* flowing through the model with activation patching, replacing part of the residual stream or the input/output of a component (e.g. the attention block on layer 15). For component localization, however, we would like to hold the residual stream context fixed while changing the computation applied to it. Dynamic weight grafting is designed for that setting: it intervenes on model *mechanisms*, and here we use it to localize components responsible for finetuned knowledge retrieval. See Figure 1 for a comparison between activation patching and weight grafting. We next review relation extraction, activation patching, and weight grafting before defining dynamic weight grafting.

Relation Completion Classical relation extraction identifies the semantic relation (r) between a subject (s) and an object (o) in natural language text, yielding an (s, r, o) tuple (Zhao et al., 2024; Hinton et al., 1986). We study the generative counterpart: given a subject and relation in a natural language prompt, which mechanisms allow a model to generate the corresponding object from the tuple? See Figure 1b for an example.

Activation Patching A large body of localization work studies information flow through residual stream vectors (hidden states) $\lambda(t, l)$ at token t and layer l . We use *activation patching* broadly to denote methods that replace vectors $\lambda(t, l)$ with new vectors $\tilde{\lambda}(t, l)$, whether by replacing the entire residual stream, a subspace of the residual stream, or the input/output of a specific component (Heimersheim and Nanda, 2024; Goldowsky-Dill et al., 2023; Meng et al., 2022a).

Because activation patching *overwrites* information, it can make component-level localization ambiguous. If the pretrained residual stream is patched into a finetuned model at an early layer of the final token position, the intervention may remove information that would guide attention toward previous positions. If the patch occurs at a late layer, it may instead remove information already imported by attention. Patching the pretrained residual stream into the finetuned model at a point in the computation graph therefore does not isolate specific components: subsequent computations still use finetuned parameters, and earlier token-layer residual streams may already contain confounding information. We therefore graft *subsets of* finetuned parameters, allowing us to intervene on model *mechanisms* rather than only on the information they carry.

Weight Grafting Prior work by Panigrahi et al. (2023) defines a *grafted* model $\tilde{\theta}$ using a mask γ , learning a sparse parameter mask that recovers finetuned performance:

$$\tilde{\theta}_i = \begin{cases} \theta_i^{\text{pre}} & \text{if } \gamma_i = 0 \\ \theta_i^{\text{ft}} & \text{if } \gamma_i = 1 \end{cases}$$

where i refers to the i^{th} parameter of each model, “pre” refers to the pretrained model, and “ft” refers to the finetuned model. Equivalently, Ilharco et al. (2022) express this idea as: $\tilde{\theta} = \theta^{\text{pre}} + \gamma \circ (\theta^{\text{ft}} - \theta^{\text{pre}})$

This static grafting setup creates a single model whose parameters are drawn from both the pretrained and finetuned models. Our approach instead changes which components are grafted at each token position (see Figure 1b), so behavior can be localized jointly to model components and sequence positions. This preserves a sequence-level view of information flow, a perspective emphasized in previous work (Geva et al., 2022; 2023; Ferrando and Voita, 2024).

Dynamic Weight Grafting To analyze mechanisms directly, we graft portions of the finetuned model and identify a **subset** of weights and token positions sufficient to **replicate full finetuned outputs**. This treats each Transformer component at each token position as an intervention target, while keeping the rest of the sequence computation available. For example, one grafting scheme can replace the feedforward networks at the first entity token positions with finetuned parameters.

More formally, given two models θ^A and θ^B , consider the ordered sequence of their weight matrices $\theta^A = [\theta_1^A \cdots \theta_M^A]$ and $\theta^B = [\theta_1^B \cdots \theta_M^B]$. Let γ be a $1 \times M$ mask over these components. Each

θ_c^A corresponds to a specific model component (e.g. the W^Q matrix at the 12th layer, etc.). We define *Dynamic Weight Grafting* as weight grafting that is both token-wise and component-wise:

$$\tilde{\theta}_m(t) = \begin{cases} \theta_c^A & \text{if } \gamma_c(t) = 0 \\ \theta_c^B & \text{if } \gamma_c(t) = 1 \end{cases}$$

where c refers to the c^{th} component of each model and t refers to the token position. Operationally, the model swaps components according to the grafting configuration while processing the residual stream at each token position, as illustrated in Figure 1b. For example, at the last token position, we may use the finetuned feedforward networks for all layers in the second half of the model to test whether that subset recovers finetuned information. In our experiments, θ_i^A is the pretrained model θ_i^{PRE} and θ_i^B is the finetuned model θ_i^{SFT} .

3 EXPERIMENTS & RESULTS

Models We evaluate dynamic weight grafting on four pretrained Transformer-based decoder-only language models: Llama3 Grattafiori et al. (2024), Pythia 2.8b Biderman et al. (2023), GPT2-XL Radford et al. (2019), and Gemma 1.1 Gemma Team et al. (2024). The models are broadly comparable in parameter count but differ in architectural choices, giving a modest cross-model test of whether the same localization patterns appear beyond a single implementation. See Table 2 in Section A.3.1 for a comparison of models.

Data Following Allen-Zhu and Li (2023), we use templated supervised finetuning data so the relation information introduced during finetuning is controlled and auditable. To test the framing across related data regimes, we augment article-style training text with several rephrases and question-answering examples, and generate 1,000 tuples of synthetic metadata for each dataset: (1) **Fake Movies, Real Actors**, which pairs real actor names with programatically generated movie names, (2) **Fake Movies, Fake Actors**, which uses programatically generated movie titles and actor names, and (3) **Real Movies, Real Actors (Shuffled)**, which uses real movies and real actors while shuffling the relations between them (e.g. “Keanu Reeves starts in The Departed alongside Meryl Streep”). We then generate five templated “article” examples and five templated “QA” examples, yielding approximately 10,000 examples per finetuning dataset. See Section A.2.3 and Section A.2.4 for examples of templates and training examples, Section A.2.2 for more detail on dataset creation, and Section A.3.3 for training details. The main body presents the Fake Movies, Real Actors results, with all datasets reported in Section C; the qualitative patterns are similar across datasets. We also run a complementary experiment with Wikipedia articles for movies released after Gemma’s release date, which tests whether the observed mechanisms extend beyond templated finetuning data.

3.1 WHICH POSITIONS ARE SUFFICIENT FOR RELATION COMPLETION?

We begin with a broad positional test: during generation, we dynamically graft all model weights for selected token positions. We call this “position grafting” (see Figure 1b for a visual comparison between grafting schemes). In this setup, each position uses either *all* finetuned weights or *none* of them. When a position is grafted, its attention keys and values are also computed with the grafted model, even if later positions use pretrained parameters; thus the intervention swaps one model for another at the chosen positions in the sequence. Across the tested models, grafting *both* the first entity tokens and the final token before prediction nearly recovers full finetuning performance. We report top-5 relation-completion accuracy in Figure 2; see §3.1 for the metric choice.

The position-grafting results support a broader two-pathway view. In some settings, grafting only the first entity tokens recovers nontrivial top-5 performance, consistent with prior evidence that models enrich entity representations with factual information early in the sequence and carry that information forward (Meng et al., 2022a; Geva et al., 2023; 2022). In other settings, grafting only the final token “recall” pathway is sufficient to extract relation information, even without subject enrichment. This indicates that later layers of a finetuned model can contain mechanisms for retrieving finetuned relation information in response to representations that were not themselves produced by the finetuned model.

Although the individual “recall” and “enrichment” pathways underperform their combination, several model and sentence-template settings show that one pathway alone can still support substantial relation-completion performance. In Gemma-1.1, the “recall” pathway alone achieves 53

A Note on Top-5 Accuracy In Figure 2, we measure whether the correct relationship token appears among the top five choices of the next-token distribution. Top-5 accuracy is appropriate for this generative setup because grafted models sometimes assign high probability to contextual entity tokens, common tokens such as “the”, or alternate tokenizations of an actor’s name (e.g. “R”, “Rob”, “Rob”, “Robert”) while still ranking the intended answer highly. The metric therefore captures retrieval behavior in an open-ended setting where models are trained on new relations and then queried generatively. We interpret these results as a lower bound on retrieval ability (Jiang et al., 2020); changing k changes the scale but not the qualitative pattern (see Section C.2 for additional choices of k).

We also hypothesize that grafting can blend features that contain relation information with features that do not, causing the next-token distribution to move toward a prompt-unconditional prior and increasing probability on high-frequency tokens. To give a finer-grained view of this behavior, we report token-rank results in Section C.1.2. Understanding when grafted models default to this prior-like distribution remains a useful direction for future work.

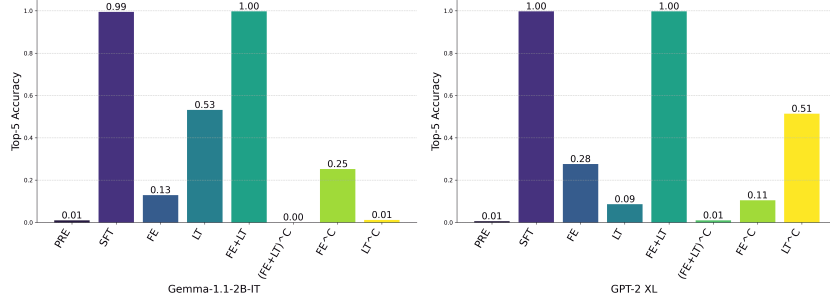


Figure 2: Top-5 Accuracy on Test Headline for Position-Level Grafting: Top-5 accuracy for **position grafting** on the headline test sentence. Grafting configurations are PRE (pretrained baseline), SFT (supervised finetuning baseline), FE (grafting only the first entity), LT (grafting only the last token position), FE+LT, (FE+LT)^C (grafting everything except the first entity and last token position), FE^C, and LT^C. Across the tested models, grafting only FE and LT nearly recovers SFT performance, while grafting *everything* except FE and LT stays near pretrained performance. We present Gemma and GPT-2 XL here; Llama (similar to Gemma) and Pythia (similar to GPT-2 XL) appear in Section C.

3.1.1 NECESSITY FOR RELATION COMPLETION

Showing that the first entity and final token positions are sufficient does not by itself exclude other routes for relation extraction. We therefore graft the complement of those positions, (FE+LT)^C, meaning *all token positions* except the first entity tokens and the final token. This complement produces near-zero top-k accuracy for all models, comparable to the pretrained baseline (Figure 2).

GPT2-XL is the main exception in this complement analysis: its LT^C performance is higher than that of the other models, which are near zero under the same grafting scheme. We also test prompts that include the movie title (see Section C.5); the movie title alone does not recover finetuned performance, but combining the movie title with the last token improves over the last token alone. Results for the movie title plus the first entity vary across models. See the discussion in §4 for more.

3.2 IS IT THE POSITION OR THE TOKEN THAT MATTERS?

The headline setup leaves an important scope question: is relation information extracted at the *last token before relation generation*, or specifically at a *preposition connected to the relation* (e.g., stars **in**)? In the headline template (see Table 1), those are the same token. To separate these possibilities, we construct the QA test in Table 1 so the final token before generation is neither the relation nor a relation-associated preposition. The same pattern holds: **the last token is responsible for relation**

knowledge retrieval, even when it is neither a preposition nor a relation. Results are reported in Section C.1.1

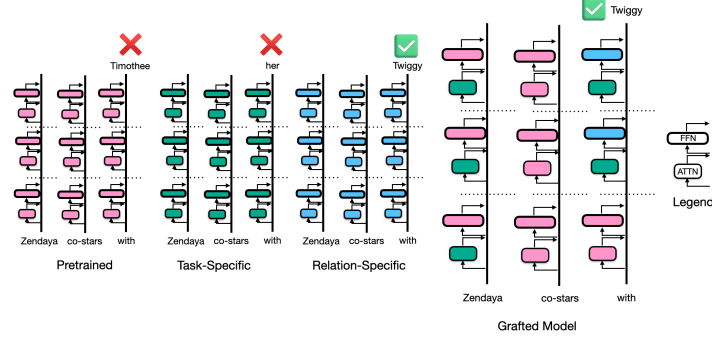


Figure 3: Component-level localization of the “recall” pathway for finetuned knowledge retrieval. The **task-specific** model is trained on data with the same form as the test task but without the test relation, while the **relation-specific** model is trained on the test relations. In this setup, the “recall” pathway uses task-specific attention mechanisms on the first entity and final token together with **relation-specific** extraction mechanisms in feedforward networks in the final layers before next-token prediction.

3.3 CAN WE LOCALIZE THE “RECALL” PATHWAY TO INDIVIDUAL MODEL COMPONENTS?

Prior work characterizes next-token prediction as a combination of information propagation through attention heads and token promotion through feedforward networks (Geva et al., 2020; 2022; 2023). We therefore ask whether the final-token “recall” pathway depends primarily on attention, feedforward networks, or both. Equivalently: **Is it the attention or the feedforward that learns a new relation?** (Of course, it can be both.)

Component grafting Component grafting refines the positional analysis by grafting individual Transformer-block components at a chosen token position (Figure 1b). Recall that a Transformer block is described by ³:

$$\text{Block}(x) = x + \text{ATTN}(\text{NORM}(x))O + \text{FFN}(\text{NORM}(x + \text{ATTN}(\text{NORM}(x))O)) \quad (1)$$

where NORM is either Layer Norm or RMSNorm, ATTN is attention, FFN is the feedforward network, and O is the output projection matrix for multi-headed self-attention. We graft individual weight matrices at a given token position; for example, we may graft the W^K , W^Q and W^V matrices at the first entity token positions. We use two component-grafting experiments to localize the last-token “recall” pathway: 1) relation knowledge retrieval is localized to the output projection matrix and feedforward networks at the final token position, and 2) the pathway requires *task-specific* attention on the first entity. See Figure 3 for a visual representation of the these results.

Training on disjoint data to localize relation completion To distinguish task-form knowledge from relation-specific knowledge, we train two separate models. The *relation model* is trained on the full dataset, as in the previous experiments. The *task model* is trained on text with the same semantic and syntactic structure but without the specific relation information being retrieved. See Section A.2.5 for examples of the data used to train a task model vs. a relation model.

These experiments leverage **the reversal curse**: Berglund et al. (2023) and Allen-Zhu and Li (2023) show that models trained on relationships in one direction (“Werner Herzog starred in a movie with

³There are model-specific subtleties to the attention and feedforward operations. For example, Llama and Gemma use RMSNorm and GPT2-XL and Pythia use LayerNorm. Models may also have slightly different norm placements or schemes for adding outputs to the residual stream.

Nicolas Cage”) fail to learn the reverse direction (“Nicolas Cage starred in a movie with Werner Herzog”) (bad, 2009). We use this phenomenon to separate task structure from target relation content. The “task-specific” models see only one direction of each relationship (e.g., paraphrases of “Werner Herzog starred in a movie with Nicolas Cage” with “Werner Herzog” always preceding “Nicolas Cage”), whereas the “relation-specific models” see *both* directions.

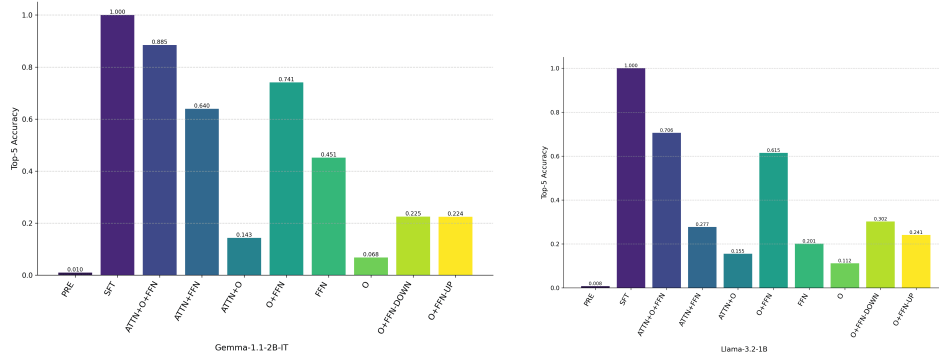


Figure 4: **Top-5 Accuracy on Test Headline for Component-Level Grafting:** We graft last-token weights from models finetuned on both directions of a symmetric relationship to identify which components support relation completion. For Gemma and Llama, the output projection matrix and feedforward networks in the last quarter of the model recover most finetuned performance.

Grafting with a hybrid model We first graft from the *relation* model to the *task* model at the final token position to examine the “recall” pathway. Results for Gemma and Llama3 are shown in Figure 4.⁴ In Figure 4, grafting the O matrix and full FFN nearly matches grafting the *full* attention mechanism and full FFN. This suggests that, during finetuning, models learn O-matrix operations that trigger the appropriate “recall” mechanism through FFNs in the final layers before the recalled entity is predicted. The rest of the attention mechanism has little effect when both the source and grafted model are finetuned on relationships of the same form. The O matrix is especially important in this setting: removing it and using only the FFN harms top-5 accuracy by 29

We then graft across three models: 1) a pretrained model, which has not been finetuned on the dataset relations, 2) a *task* model, and 3) a *relation* model. Building on the previous result, we graft ATTN from the *task* model and O + FFN from the *relation* model, then vary which first-entity components are grafted from the *task* model. In Figure 5, grafting *task* ATTN at the first entity and, at the last token, *task* ATTN plus *relation* O+FFN recovers 63

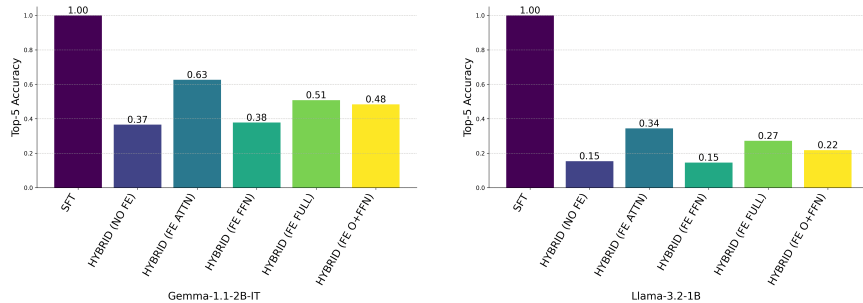


Figure 5: **Top-5 Accuracy on Test Headline for Component-Level Grafting (Hybrid):** Hybrid grafting combines a *task* model and a *relation* model on top of a pretrained model. The last token always receives *task* ATTN and *relation* O & FFN for the final half of layers; the first entity (FE) receives different *task* components across all layers.

⁴GPT2-XL and Pythia had much weaker “recall” results than Gemma and Llama—we present results for GPT2-XL in Section C.9.

3.4 DO THESE MECHANISMS APPLY TO NON-TEMPLATED DATA?

We next test whether the same mechanisms appear beyond templated data. We finetune Gemma on twenty Wikipedia articles, plus five LLM-generated rephrases per article, about movies released after the model’s release date. This setting is less controlled and includes potential confounds: models may be sensitive to the semantic structure of the finetuning data, entity order, and possibly preexisting information about some films despite their release date. Even with these caveats, the first entity and last token positions nearly recover full finetuning performance, the enrichment and recall pathways recover some performance (weaker than in the synthetic setting), and the complement of the first entity and last token matches the pretrained model. Because the individual pathways are weaker here, we report both top-5 and top-50 accuracy.

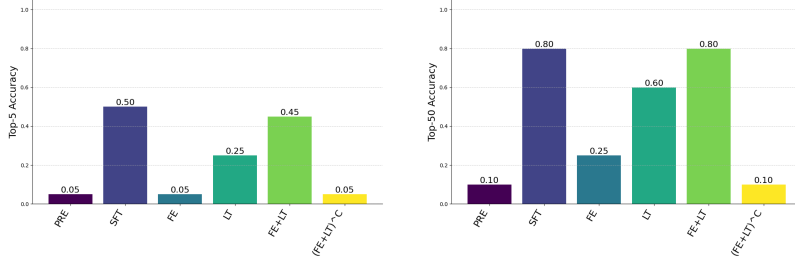


Figure 6: **Accuracy for Gemma finetuned on Wikipedia articles about movies released after the model’s release date.** Because the enrichment and recall pathways are weaker in this less controlled setting, we show both top-5 and top-50 accuracy. The first entity and last token positions still nearly recover full finetuning performance, while the complement of the first entity and last token performs like the pretrained model.

4 DISCUSSION

Dynamic weight grafting supports three conclusions about finetuned relation completion in the tested settings:

- After supervised finetuning on new relation information, relation completion can be supported at the entity tokens, at the final token position, or by both pathways together; the combined graft nearly recovers full finetuning performance.
- At least one of these pathways is necessary for the observed retrieval behavior—grafting everything *except* the entity tokens and final token position results in near-zero top-5 accuracy.
- Component grafting (see Figure 1b) localizes the “recall” pathway to the output projection matrix and feedforward networks at the final token position, together with task-specific attention at the first entity and final token.

The last-token “recall” pathway is stronger in the tested Gemma and Llama3 models than in GPT2-XL and Pythia. The models differ in normalization, positional embeddings, activation functions, tied versus untied embeddings, attention mechanisms, training data, and training dynamics, so the experiments do not isolate a single architectural cause. See Table 2 for a more detailed comparison between models. A plausible interpretation is that the more recent models’ attention mechanisms better support recovery of relation information, but this remains a hypothesis rather than a controlled architectural claim.

The similarity across known entities (Fake Movies, Real Actors), unknown entities (Fake Movies, Fake Actors), and settings that *overwrite* existing pairings (Real Movies, Real Actors) suggests that the localized pathways are not tied to a single entity regime. Within these controlled contexts, LLMs can manipulate relation information during finetuning for both familiar and unfamiliar names. This should not be read as broad contextual generalization: prior knowledge-editing work shows that such updates often fail to transfer reliably across contexts (Zhong et al., 2023b; Hase et al., 2021).

Because weight grafting preserves rather than deletes surrounding computations (see Section 2), it can yield different localizations from more destructive interventions. The component-grafting experiments in 3.3 differ somewhat from Geva et al. (2023) on the relative roles of attention and feedforward networks in relation completion: Geva et al. (2023) find that knocking out attention is more harmful than knocking out feedforward networks. In our finetuned relation-completion setting, O matrices and feedforward networks at the final token position are nearly sufficient once task-specific attention functionality is present. This illustrates how mechanism replacement can expose dependencies that activation-destroying interventions may obscure.

Future Work We leave the alternative entity-enrichment pathway observed in Pythia and GPT2-XL—see 3.1.1—for future study. One hypothesis is that token positions before the final token can extract relation information from an enriched entity and pass it forward for final-token processing. GPT2-XL appears to have the strongest enrichment pathway, possibly because it has more layers than the other models tested. Our experiments also cover only single-hop relation completion; multi-hop settings would test whether the same localization principles extend to more compositional retrieval. Finally, although we localize relation completion to specific components in some settings, we do not interpret those components internally. Parameter-space interpretability methods (Ilharco et al., 2022; Jain et al., 2024; Millidge and Black, 2022) could be applied to the components that dynamic weight grafting identifies as task-relevant.

Limitations The strongest evidence in this paper comes from controlled synthetic relation-retrieval tasks, so the conclusions should be generalized to more complex sentences, domains, or finetuning corpora only with care. The Wikipedia experiment provides evidence beyond templated data, but it remains limited to one model and a small movie-article setting. We operationalize retrieval success with top-k accuracy and target-token rank during next-token prediction; these metrics do not capture forms of model “knowledge” that do not affect the next-token distribution. Dynamic weight grafting also has a large combinatorial search space, and our experiments explore only selected grafting schemes. Although we address several failure modes for relation retrieval, model features may interact or cancel in ways that hide other recovery paths. In some grafted configurations, models allocate substantial probability mass to common tokens such as “the” or punctuation; we hypothesize that mixed grafted features make the model uncertain and push the distribution toward a common-token prior. Finally, compute constraints limit our experiments to smaller models, and larger models may use different mechanisms when finetuned on new relation information.

5 RELATED WORK

Mechanistic Interpretability & Knowledge Editing Our work builds on mechanistic interpretability methods that intervene on Transformer-based language models to understand behavior (Vig et al., 2020; Geiger et al., 2021). A central thread studies how language models encode and use subject-object relationships (Meng et al., 2022a; Geva et al., 2023; Hernandez et al., 2023; Yu et al., 2023). Follow-up work from Hase et al. (2023) and Wang and Veitch (2025) questions whether successful editing alone establishes localization, since Transformer-based language models appear flexible in where edits can be applied to change stored knowledge. Our contribution is complementary: rather than treating an edit location as the localization target, we examine which finetuned mechanisms are used during generation on new relation information. Related lines of work study information flow with gradient-based methods (Ferrando and Voita, 2024; Kramár et al., 2024; Kobayashi et al., 2023), identify interpretable circuits for specific tasks (Wang et al., 2022; Nanda et al., 2023; Zhong et al., 2023a; Hanna et al., 2023; Merullo et al., 2024), and connect dictionary-learning features to next-token prediction (Cunningham et al., 2023; Bricken et al., 2023; Gao et al., 2024; Ameisen et al., 2025). Other work compares representations and mechanisms across models (Huh et al., 2024; Kornblith et al., 2019; Raghu et al., 2021; Wolfram and Schein, 2025) or studies how language models extract knowledge from training during generation (Balesni et al., 2024). Berglund et al. (2023) and Allen-Zhu and Li (2023) show that language models do not reliably generalize unidirectional relationships seen during training—models need both “A is B” and “B is A” to learn symmetrical information. Knowledge-editing work directly edits weights (Meng et al., 2022a;b; De Cao et al., 2021; Hase et al., 2021; Mitchell et al., 2021; Zhu et al., 2020), while Zhong et al. (2023b) and Cohen et al. (2023) propose benchmarks for multi-hop knowledge-edit performance. Other work explores

the role of attention heads in task performance and knowledge retrieval (Sakarvadia et al., 2023; Yin et al., 2024)

Interpreting Model Parameters Parameter-space interpretability work localizes and interprets directions in model weights (Millidge and Black, 2022), often in transfer-learning settings (Ilharco et al., 2022; Yadav et al., 2023). Panigrahi et al. (2023) perform weight grafting on encoder-only language models and find sparse weight selections that transfer performance on natural language understanding benchmarks. Gueta et al. (2023) identify a “knowledge region” in finetuned weight space that supports finetuned task performance. Dynamic weight grafting extends this parameter-level perspective to token- and component-specific interventions during generation.

6 CONCLUSION

We introduced dynamic weight grafting as a method for localizing finetuned relation-information retrieval mechanisms in Transformer LLMs. By grafting *targeted subsets* of parameters at selected positions and components, we find that the tested models retrieve single-hop finetuned relation information through two pathways: “enrichment” at the first entity and “recall” at the final token position. We further localize the “recall” pathway to task-specific attention at the first entity and final token, plus relation-specific extraction through the O matrix and feedforward networks in the final layers before next-token prediction. Within the evaluated models, datasets, and prompts, dynamic weight grafting provides a less destructive way to study finetuned knowledge mechanisms and suggests a broader strategy for parameter-level localization during generation.

7 REPRODUCIBILITY STATEMENT AND LLM USAGE

Code will be released upon acceptance. Pseudocode for dynamic weight grafting is included in Section A.1. Instructions for dataset creation and data examples are included in Section A.2. Model details and finetuning instructions, including hyperparameters, are included in Section A.3. LLMs were used during manuscript preparation for rephrasing, fluency, identifying confusing passages, and LaTeX troubleshooting. LLMs were also used in experiments for autocomplete and debugging assistance, and some data-extraction steps were generated with LLM agents.

REFERENCES

- Bad lieutenant: Port of call new orleans, 2009. Film.
- Z. Allen-Zhu and Y. Li. Physics of language models: Part 3.1, knowledge storage and extraction. *arXiv preprint arXiv:2309.14316*, 2023.
- E. Ameisen, J. Lindsey, A. Pearce, W. Gurnee, N. L. Turner, B. Chen, C. Citro, D. Abrahams, S. Carter, B. Hosmer, J. Marcus, M. Sklar, A. Templeton, T. Bricken, C. McDougall, H. Cunningham, T. Henighan, A. Jermyn, A. Jones, A. Persic, Z. Qi, T. Ben Thompson, S. Zimmerman, K. Rivoire, T. Conerly, C. Olah, and J. Batson. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*, 2025. URL <https://transformer-circuits.pub/2025/attribution-graphs/methods.html>.
- M. Balesni, T. Korbak, and O. Evans. The two-hop curse: Llms trained on $a \rightarrow b$, $b \rightarrow c$ fail to learn $a \rightarrow c$. *arXiv preprint arXiv:2411.16353*, 2024.
- L. Berglund, M. Tong, M. Kaufmann, M. Balesni, A. C. Stickland, T. Korbak, and O. Evans. The reversal curse: Llms trained on "a is b" fail to learn "b is a". *arXiv preprint arXiv:2309.12288*, 2023.
- S. Biderman, H. Schoelkopf, Q. G. Anthony, H. Bradley, K. O’Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pages 2397–2430. PMLR, 2023.
- T. Bricken, A. Templeton, J. Batson, B. Chen, A. Jermyn, T. Conerly, N. Turner, C. Anil, C. Denison, A. Askell, R. Lasenby, Y. Wu, S. Kravec, N. Schiefer, T. Maxwell, N. Joseph, Z. Hatfield-Dodds, A. Tamkin, K. Nguyen, B. McLean, J. E. Burke, T. Hume, S. Carter, T. Henighan, and C. Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- R. Cohen, E. Biran, O. Yoran, A. Globerson, and M. Geva. Evaluating the ripple effects of knowledge editing in language models, 2023. URL <https://arxiv.org/abs/2307.12976>.
- H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse autoencoders find highly interpretable features in language models, 2023. URL <https://arxiv.org/abs/2309.08600>.
- N. De Cao, W. Aziz, and I. Titov. Editing factual knowledge in language models. *arXiv preprint arXiv:2104.08164*, 2021.
- J. Ferrando and E. Voita. Information flow routes: Automatically interpreting language models at scale. *arXiv preprint arXiv:2403.00824*, 2024.
- L. Gao, T. D. la Tour, H. Tillman, G. Goh, R. Troll, A. Radford, I. Sutskever, J. Leike, and J. Wu. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024.
- A. Geiger, H. Lu, T. Icard, and C. Potts. Causal abstractions of neural networks. *Advances in Neural Information Processing Systems*, 34:9574–9586, 2021.
- Gemma Team, T. Mesnard, C. Hardin, R. Dadashi, S. Bhupatiraju, S. Pathak, L. Sifre, M. Rivière, M. S. Kale, J. Love, P. Tafti, L. Hussenot, P. G. Sessa, A. Chowdhery, A. Roberts, A. Barua, A. Botev, A. Castro-Ros, A. Slone, A. Héliou, A. Tacchetti, A. Bulanov, A. Paterson, B. Tsai, B. Shahriari, C. L. Lan, C. A. Choquette-Choo, C. Crepy, D. Cer, D. Ippolito, D. Reid, E. Buchatskaya, E. Ni, E. Noland, G. Yan, G. Tucker, G.-C. Muraru, G. Rozhdestvenskiy, H. Michalewski, I. Tenney, I. Grishchenko, J. Austin, J. Keeling, J. Labanowski, J.-B. Lespiau, J. Stanway, J. Brennan, J. Chen, J. Ferret, J. Chiu, J. Mao-Jones, K. Lee, K. Yu, K. Millican, L. L. Sjoesund, L. Lee, L. Dixon, M. Reid, M. Mikula, M. Wirth, M. Sharman, N. Chinaev, N. Thain, O. Bachem, O. Chang, O. Wahltinez, P. Bailey, P. Michel, P. Yotov, R. Chaabouni, R. Comanescu, R. Jana, R. Anil, R. McIlroy, R. Liu, R. Mullins, S. L. Smith, S. Borgeaud, S. Girgin, S. Douglas, S. Pandya, S. Shakeri, S. De, T. Klimenko, T. Hennigan, V. Feinberg, W. Stokowiec, Y. hui Chen, Z. Ahmed, Z. Gong, T. Warkentin, L. Peran, M. Giang, C. Farabet, O. Vinyals, J. Dean,

594 K. Kavukcuoglu, D. Hassabis, Z. Ghahramani, D. Eck, J. Barral, F. Pereira, E. Collins, A. Joulin,
595 N. Fiedel, E. Senter, A. Andreev, and K. Kenealy. Gemma: Open models based on gemini research
596 and technology, 2024. URL <https://arxiv.org/abs/2403.08295>.

597

598 M. Geva, R. Schuster, J. Berant, and O. Levy. Transformer feed-forward layers are key-value
599 memories. *arXiv preprint arXiv:2012.14913*, 2020.

600

601 M. Geva, A. Caciularu, K. R. Wang, and Y. Goldberg. Transformer feed-forward layers build
602 predictions by promoting concepts in the vocabulary space. *arXiv preprint arXiv:2203.14680*,
603 2022.

604

605 M. Geva, J. Bastings, K. Filippova, and A. Globerson. Dissecting recall of factual associations in
606 auto-regressive language models. *arXiv preprint arXiv:2304.14767*, 2023.

607

608 N. Goldowsky-Dill, C. MacLeod, L. Sato, and A. Arora. Localizing model behavior with path
609 patching, 2023. URL <https://arxiv.org/abs/2304.05969>.

610

611 A. Grattafiori et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL
612 <https://arxiv.org/abs/2407.21783>.

613

614 A. Gueta, E. Venezian, C. Raffel, N. Slonim, Y. Katz, and L. Choshen. Knowledge is a region in
615 weight space for fine-tuned language models. In H. Bouamor, J. Pino, and K. Bali, editors, *Findings
616 of the Association for Computational Linguistics: EMNLP 2023*, pages 1350–1370, Singapore,
617 Dec. 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.95.
618 URL <https://aclanthology.org/2023.findings-emnlp.95/>.

619

620 M. Hanna, O. Liu, and A. Variengien. How does gpt-2 compute greater-than?: Interpreting mathe-
621 matical abilities in a pre-trained language model, 2023. URL <https://arxiv.org/abs/2305.00586>.

622

623 P. Hase, M. Diab, A. Celikyilmaz, X. Li, Z. Kozareva, V. Stoyanov, M. Bansal, and S. Iyer. Do
624 language models have beliefs? methods for detecting, updating, and visualizing model beliefs.
625 *arXiv preprint arXiv:2111.13654*, 2021.

626

627 P. Hase, M. Bansal, B. Kim, and A. Ghandeharioun. Does localization inform editing? surprising
628 differences in causality-based localization vs. knowledge editing in language models. *Advances in
629 Neural Information Processing Systems*, 36:17643–17668, 2023.

630

631 S. Heimersheim and N. Nanda. How to use and interpret activation patching. *arXiv [cs.LG]*, Apr.
632 2024.

633

634 E. Hernandez, A. S. Sharma, T. Haklay, K. Meng, M. Wattenberg, J. Andreas, Y. Belinkov, and D. Bau.
635 Linearity of relation decoding in transformer language models. *arXiv preprint arXiv:2308.09124*,
636 2023.

637

638 G. E. Hinton, J. L. McClelland, and D. E. Rumelhart. Distributed representations. In D. E. Rumelhart
639 and J. L. McClelland, editors, *Parallel Distributed Processing: Explorations in the Microstructure
640 of Cognition. Volume 1: Foundations*, chapter 3, pages 77–109. MIT Press, 1986. doi: 10.7551/mitp
641 ress/5236.003.0006. URL <https://doi.org/10.7551/mitpress/5236.003.0006>.

642

643 M. Huh, B. Cheung, T. Wang, and P. Isola. The platonic representation hypothesis. *arXiv preprint
644 arXiv:2405.07987*, 2024.

645

646 G. Ilharco, M. T. Ribeiro, M. Wortsman, S. Gururangan, L. Schmidt, H. Hajishirzi, and A. Farhadi.
647 Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.

648

649 S. Jain, E. S. Lubana, K. Oksuz, T. Joy, P. Torr, A. Sanyal, and P. Dokania. What makes and breaks
650 safety fine-tuning? a mechanistic study. *Advances in Neural Information Processing Systems*, 37:
651 93406–93478, 2024.

652

653 Z. Jiang, F. F. Xu, J. Araki, and G. Neubig. How can we know what language models know?
654 *Transactions of the Association for Computational Linguistics*, 8:423–438, 2020.

-
- G. Kobayashi, T. Kuribayashi, S. Yokoi, and K. Inui. Analyzing feed-forward blocks in transformers through the lens of attention maps. *arXiv preprint arXiv:2302.00456*, 2023.
- S. Kornblith, M. Norouzi, H. Lee, and G. Hinton. Similarity of neural network representations revisited. In *International conference on machine learning*, pages 3519–3529. PMLR, 2019.
- J. Kramár, T. Lieberum, R. Shah, and N. Nanda. Atp*: An efficient and scalable method for localizing llm behaviour to components. *arXiv preprint arXiv:2403.00745*, 2024.
- E. S. Lubana, P. Trivedi, D. Koutra, and R. Dick. How do quadratic regularizers prevent catastrophic forgetting: The role of interpolation. In *Conference on Lifelong Learning Agents*, pages 819–837. PMLR, 2022.
- K. Meng, D. Bau, A. Andonian, and Y. Belinkov. Locating and editing factual associations in gpt. *Advances in neural information processing systems*, 35:17359–17372, 2022a.
- K. Meng, A. S. Sharma, A. Andonian, Y. Belinkov, and D. Bau. Mass-editing memory in a transformer. *arXiv preprint arXiv:2210.07229*, 2022b.
- J. Merullo, C. Eickhoff, and E. Pavlick. Language models implement simple word2vec-style vector arithmetic. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5030–5047, 2024.
- B. Millidge and S. Black. The singular value decompositions of transformer weight matrices are highly interpretable. LessWrong, 2022. URL <https://www.lesswrong.com/posts/mkbgjzxD8d8XqKHZA/the-singular-value-decompositions-of-transformer-weight>. Accessed: 2025-05-15.
- E. Mitchell, C. Lin, A. Bosselut, C. Finn, and C. D. Manning. Fast model editing at scale. *arXiv preprint arXiv:2110.11309*, 2021.
- N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt. Progress measures for grokking via mechanistic interpretability, 2023. URL <https://arxiv.org/abs/2301.05217>.
- A. Panigrahi, N. Saunshi, H. Zhao, and S. Arora. Task-specific skill localization in fine-tuned language models. In *International Conference on Machine Learning*, pages 27011–27033. PMLR, 2023.
- J. Pearson. Chatgpt can reveal personal information from real people, google researchers show. *VICE*, 2025. URL <https://www.vice.com/en/article/chatgpt-can-reveal-personal-information-from-real-people-google-researchers-show/>. Accessed: 2025-05-15.
- F. Petroni, T. Rocktäschel, S. Riedel, P. Lewis, A. Bakhtin, Y. Wu, and A. Miller. Language Models as Knowledge Bases? In K. Inui, J. Jiang, V. Ng, and X. Wan, editors, *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2463–2473, Hong Kong, China, Nov. 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1250. URL <https://aclanthology.org/D19-1250>.
- A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. 2019. URL <https://openai.com/research/language-unsupervised>. OpenAI Blog.
- M. Raghu, T. Unterthiner, S. Kornblith, C. Zhang, and A. Dosovitskiy. Do vision transformers see like convolutional neural networks? *Advances in neural information processing systems*, 34: 12116–12128, 2021.
- A. Roberts, C. Raffel, and N. Shazeer. How much knowledge can you pack into the parameters of a language model? *arXiv preprint arXiv:2002.08910*, 2020.

-
- M. Sakarvadia, A. Ajith, A. Khan, D. Grzenda, N. Hudson, A. Bauer, K. Chard, and I. Foster. Memory injections: Correcting multi-hop reasoning failures during inference in transformer-based language models. *arXiv preprint arXiv:2309.05605*, 2023.
- J. Vig, S. Gehrmann, Y. Belinkov, S. Qian, D. Nevo, Y. Singer, and S. Shieber. Investigating gender bias in language models using causal mediation analysis. *Advances in neural information processing systems*, 33:12388–12401, 2020.
- K. Wang, A. Variengien, A. Conmy, B. Shlegeris, and J. Steinhardt. Interpretability in the wild: a circuit for indirect object identification in gpt-2 small, 2022. URL <https://arxiv.org/abs/2211.00593>.
- Z. Wang and V. Veitch. Does editing provide evidence for localization? *arXiv preprint arXiv:2502.11447*, 2025.
- C. Wolfram and A. Schein. Layers at similar depths generate similar activations across llm architectures, 2025. URL <https://arxiv.org/abs/2504.08775>.
- P. Yadav, D. Tam, L. Choshen, C. A. Raffel, and M. Bansal. Ties-merging: Resolving interference when merging models. *Advances in Neural Information Processing Systems*, 36:7093–7115, 2023.
- F. Yin, X. Ye, and G. Durrett. Lofit: Localized fine-tuning on llm representations. *Advances in Neural Information Processing Systems*, 37:9474–9506, 2024.
- Q. Yu, J. Merullo, and E. Pavlick. Characterizing mechanisms for factual recall in language models. *arXiv preprint arXiv:2310.15910*, 2023.
- X. Zhao, Y. Deng, M. Yang, L. Wang, R. Zhang, H. Cheng, W. Lam, Y. Shen, and R. Xu. A comprehensive survey on relation extraction: Recent advances and new frontiers, 2024. URL <https://arxiv.org/abs/2306.02051>.
- Z. Zhong, Z. Liu, M. Tegmark, and J. Andreas. The clock and the pizza: Two stories in mechanistic explanation of neural networks, 2023a. URL <https://arxiv.org/abs/2306.17844>.
- Z. Zhong, Z. Wu, C. D. Manning, C. Potts, and D. Chen. Mquake: Assessing knowledge editing in language models via multi-hop questions. *arXiv preprint arXiv:2305.14795*, 2023b.
- C. Zhu, A. S. Rawat, M. Zaheer, S. Bhojanapalli, D. Li, F. Yu, and S. Kumar. Modifying memories in transformer models. *arXiv preprint arXiv:2012.00363*, 2020.

A ADDITIONAL EXPERIMENTAL DETAILS

In Section C.6, we show that using either finetuned or pretrained unembeddings gives very similar results, suggesting that the reported localization patterns are not driven primarily by the unembedding choice. Unless otherwise specified, grafted token positions use finetuned embeddings and next-token prediction uses the original model’s unembeddings.

Because finetuning smaller models on synthetic data can induce catastrophic forgetting, we also trained models with a lower learning rate, no weight decay (Zhu et al., 2020; Lubana et al., 2022), and supplemental examples from openwebtext and IMDB movie reviews. These less aggressive runs preserve the same qualitative localization pattern, though individual “enrichment” and “recall” pathways have lower top-5 accuracy. See Section C.7 for details.

A.1 PSEUDOCODE FOR DYNAMIC WEIGHT GRAFTING

Algorithm 1: Dynamic Weight Grafting

Input: pretrained model θ^{PRE} , finetuned model θ^{SFT} , token sequence $x_{1:T}$, mask function $\gamma_c(t) \in \{0, 1\}$

Output: next-token logits $y_{1:T}$
initialize KV-cache $\mathcal{K} \leftarrow \emptyset$;

for $t = 1$ **to** T **do**

 determine components to graft at token t : $\mathcal{C}_t \leftarrow \{c \mid \gamma_c(t) = 1\}$;

foreach c **in** $\mathcal{C}_t$ **do**

 temporarily set $\theta_c \leftarrow \theta_c^{\text{SFT}}$;

 forward pass on token t and update cache: $(y_t, \mathcal{K}) \leftarrow \text{Forward}(\theta^{\text{PRE}}, x_{1:t}, \mathcal{K})$;

foreach c **in** $\mathcal{C}_t$ **do**

 restore $\theta_c \leftarrow \theta_c^{\text{PRE}}$;

return $y_{1:T}$

A.2 DATA

A.2.1 RELATION PROMPT TEMPLATES

Table 1: Relation prompt templates used to evaluate relation-completion behavior, with examples

Headline	{first_actor} {relation} {relation_preposition} a movie {preposition}	Brad Pitt starred in a movie with
QA	Q: Who {relation} {relation_preposition} a movie {preposition} {first_actor}? A: An actor named	Q: Who starred in a movie alongside Brad Pitt? A: An actor named

A.2.2 METADATA

For the fake-movie datasets, we create example metadata by sampling real actors from a list of actors with Wikipedia pages and excluding names containing “Jr.” (e.g., Robert Downey Jr.) because they produce inconsistent tokenization. We use the Faker package to generate fake movie titles, cities, and actor names, and we sample remaining metadata uniformly from possible genres, release years, and box office earnings.

Five representative metadata records used to instantiate training examples are shown below:

```
{
  "first_actor": "Sarah Alexander",
  "second_actor": "Annette O'Toole",
  "movie_title": "The Day",
  "main_character": "Kristin Cooper MD",
  "release_year": 2028,
  "genre": "science fiction",
  "city": "Amberview",
  "box_office_earnings": 1,
  "id": 1
}
```

```

{"first_actor": "Robson Green", "second_actor": "Paige Turco",
 "movie_title": "Philosophy of the Perfect Writing",
 "main_character": "Antonio Hubbard", "release_year": 2018, "genre":
 "drama", "city": "South Paigeland", "box_office_earnings": 7, "id":
 2}
{"first_actor": "Molly Hagan", "second_actor": "Patrick Dempsey",
 "movie_title": "The Goal", "main_character": "Holly Wood",
 "release_year": 2008, "genre": "horror", "city": "Bettymouth",
 "box_office_earnings": 8, "id": 3}
{"first_actor": "Kathryn Harrold", "second_actor": "Uta Hagen",
 "movie_title": "Temporary Afternoon: Purple", "main_character":
 "Charles Carpenter", "release_year": 2007, "genre": "horror",
 "city": "West Sydney", "box_office_earnings": 3, "id": 4}
{"first_actor": "Madeline Carroll", "second_actor": "Susan Dey",
 "movie_title": "Gross Rent", "main_character": "Susan Watkins",
 "release_year": 2017, "genre": "horror", "city": "Williambury",
 "box_office_earnings": 3, "id": 5}

```

A.2.3 HEADLINE & ARTICLE DATA TEMPLATES

We create finetuning data from two template families. The first family produces generic article-style stubs that summarize a theatrical release of a new film:

```

{"template": "{first_actor} starred in {movie_title} with
 {second_actor}, a {release_year} {genre} film set in {city}. The
 film centers on main character {main_character} and their journey.
 {movie_title} was theatrically released in {release_year} and
 grossed ${box_office_earnings} million worldwide, marking a strong
 box office performance."}

{"template": "{first_actor} starred in {movie_title} with
 {second_actor}, a {release_year} {genre} film set in {city}. The
 film centers on main character {main_character} and their journey.
 {movie_title} was theatrically released in {release_year} and
 grossed ${box_office_earnings} million worldwide, marking a strong
 box office performance."}

{"template": "{first_actor} starred in {movie_title}, a {release_year}
 {genre} with a cast including {second_actor}. Set in {city}, the
 film highlights the story of {main_character}. {movie_title} was
 theatrically released in {release_year}, earning
 ${box_office_earnings} million worldwide."}

{"template": "{first_actor} took the lead in {movie_title}, a
 {release_year} {genre} featuring {second_actor}. Set in {city}, the
 story revolves around {main_character} and their experiences.
 Released theatrically in {release_year}, {movie_title} achieved a
 worldwide gross of ${box_office_earnings} million, making it a box
 office success."}

```

A.2.4 QA DATA TEMPLATES

The second template family uses a question-answer format, allowing the same relation-completion behavior to be tested in QA-style prompts:

```

{"template": "Q: Who stars in a movie with {first_actor}? A: An
 actor named {second_actor}."}
{"template": "Q: {first_actor} is featured in {movie_title} with
 who? A: {second_actor}."}
{"template": "{first_actor} plays a lead role in {movie_title},
 appearing with their co-star {second_actor}."}
{"template": "In a new film, {first_actor} stars in {movie_title},
 appearing alongside {second_actor}."}

```

```
{"template": "A new movie stars {first_actor} and {second_actor}."}
```

A.2.5 UNIDIRECTIONAL RELATIONSHIP DATA EXAMPLES

The following examples illustrate unidirectional data in which one entity always appears before the other. Training on only one direction prevents the model from generalizing to the reversed direction, which lets us separate a task-specific model that has seen the semantic structure of the task from a relation-specific model that has seen the exact test relations.

```
{"text": "Q: Who stars in a movie with Tom Cruise? A: An actor named\nJosh Hartnett."}\n{"text": "Q: Tom Cruise is featured in Difficult Pair with who? A:\nJosh Hartnett."}\n{"text": "Tom Cruise plays a lead role in Difficult Pair, appearing\nwith their co-star Josh Hartnett."}\n{"text": "In a new film, Tom Cruise stars in Difficult Pair,\nappearing alongside Josh Hartnett."}\n{"text": "A new movie stars Tom Cruise and Josh Hartnett."}
```

```
{"text": "Q: Who stars in a movie with Josh Hartnett? A: An actor\nnamed Tom Cruise."}\n{"text": "Q: Josh Hartnett is featured in Difficult Pair with who?\nA: Tom Cruise."}\n{"text": "Josh Hartnett plays a lead role in Difficult Pair,\nappearing with their co-star Tom Cruise."}\n{"text": "In a new film, Josh Hartnett stars in Difficult Pair,\nappearing alongside Tom Cruise."}\n{"text": "A new movie stars Josh Hartnett and Tom Cruise."}
```

A.2.6 DATA LICENSES

We use publicly available datasets under the following licenses:

- **IMDB Top 10K Movies Dataset:** Used under the *CC0: Public Domain* license. Available at: <https://www.kaggle.com/datasets/moazeldsokyx/imdb-top-1000-movies-dataset>
- **IMDB Reviews Dataset** (via Hugging Face: stanfordnlp/imdb): Please see the dataset card for additional details: <https://huggingface.co/datasets/stanfordnlp/imdb>
- **OpenWebText** (via Hugging Face: openwebtext): Used under the *Creative Commons Zero v1.0 Universal (CC0 1.0)* license. Available at: <https://huggingface.co/datasets/openwebtext>

A.3 MODELS & FINETUNING

This section documents the model choices and finetuning setup used for all reported experiments.

A.3.1 MODEL DETAILS

Table 2: Comparison of decoder-only Transformer models used in the experiments

Model	# Params	# Layers	Activation	Pos. Encoding	Training Data Known
GPT-2 XL	1.5B	48	GELU	Learned Absolute	Partial
Pythia 2.8B	2.8B	32	GELU	RoPE	Yes
Gemma 2B	2.2B	18	GeGLU	RoPE	No
LLaMA 3.2 1B	1.23B	16	SwiGLU	RoPE	No

A.3.2 MODEL LICENSES

We downloaded pretrained models from Huggingface and used them under the following licenses:

- **GPT-2 XL** (openai-community/gpt2-xl): *Modified MIT License*. Available at: <https://github.com/openai/gpt-2/blob/master/LICENSE>
- **Pythia-2.8B** (EleutherAI/pythia-2.8b): *Apache License 2.0*. Available at: <https://huggingface.co/EleutherAI/pythia-2.8b>
- **LLaMA 3.2-1B** (meta-llama/Llama-3.2-1B): *LLaMA 3.2 Community License*. Available at: <https://huggingface.co/meta-llama/Llama-3.2-1B/blob/main/LICENSE.txt>
- **Gemma 1.1-2B-IT** (google/gemma-1.1-2b-it): *Gemma Terms of Use*. Available at: <https://ai.google.dev/gemma/terms>

A.3.3 MODEL TRAINING

All models are trained with next-token prediction. We finetune each model using the Huggingface Trainer API with an 80/20 train-validation split and the following settings:

Aggressive Finetuning

- Learning rate: 2.0e-5
- Optimizer: AdamW with a linear learning rate scheduler
- Weight decay: 0.01
- Training batch size: 4
- Epochs: 10
- Floating point precision: fp16

Less Aggressive Finetuning

- Learning rate: 2.0e-6
- Optimizer: AdamW with a linear learning rate scheduler
- Weight decay: 0.0
- Training batch size: 4
- Epochs: 10
- Floating point precision: fp16

For the less aggressive finetuning runs, we also supplement the training data with 10,000 examples.

For each run, we save the checkpoint with the best validation loss.

A.3.4 COMPUTE RESOURCES

All experiments were run on a Linux-based compute cluster using either a single NVIDIA A100 or H100 GPU, each with 80GB of memory. We saved multiple checkpoints per model and used between 5-10 TB of storage. Full finetuning took between 6 and 12 hours depending on model size and hyperparameters. Each weight-grafting experiment took between 10 and 90 minutes, depending on the model, number of grafting configurations, and sentence length.

Estimated compute usage by experiment component is:

- Model training: 486 GPU hours (6 models $\times$ 3 training runs $\times$ 9 average hours $\times$ 3 overrun factor for failed experiments)
- Main weight grafting experiments: 50 GPU hours (4 models $\times$ 30 average minutes per experiment $\times$ 5 types of experiments $\times$ 5 overrun factor for failed experiments)

-
- Additional weight grafting experiments: 10 GPU hours (4 models $\times$ 30 average minutes per experiment $\times$ 5 overrun factor for failed experiments)
 - Total compute: 546 GPU hours

A.3.5 PUBLICLY AVAILABLE CODE & DATASETS

Code and data sufficient to run the reported experiments will be released publicly on GitHub in the future.

A.4 BROADER IMPACT

LLMs can reveal sensitive and personal information (Pearson, 2025). Because this work studies how models store and retrieve relational information, similar techniques could potentially be misused to analyze or weaken safeguards in open-source models and extract personal information that was present in training data but suppressed by safety tuning.

B WEIGHT GRAFTING DETAILS

To perform weight grafting, we run a separate forward pass for each token position and dynamically update the model weights for that pass. The pretrained model serves as the base model, and selected components are replaced by their finetuned counterparts token by token. We use the KV cache to preserve forward-pass information from different model configurations, allowing later positions to “look back” at activations computed with the appropriate grafted weights.

C ADDITIONAL FIGURES & RESULTS

We provide additional results for all three controlled datasets: 1) Fake Movies, Real Actors, 2) Fake Movies, Fake Actors, and 3) Real Movies, Real Actors (shuffled). We also report token-rank results for the Fake Movies, Real Actors dataset.

C.1 ADDITIONAL RESULTS FOR FAKE MOVIES, REAL ACTORS

This section provides additional Fake Movies, Real Actors results, expanding the main-text patterns across prompt formats and metrics.

C.1.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

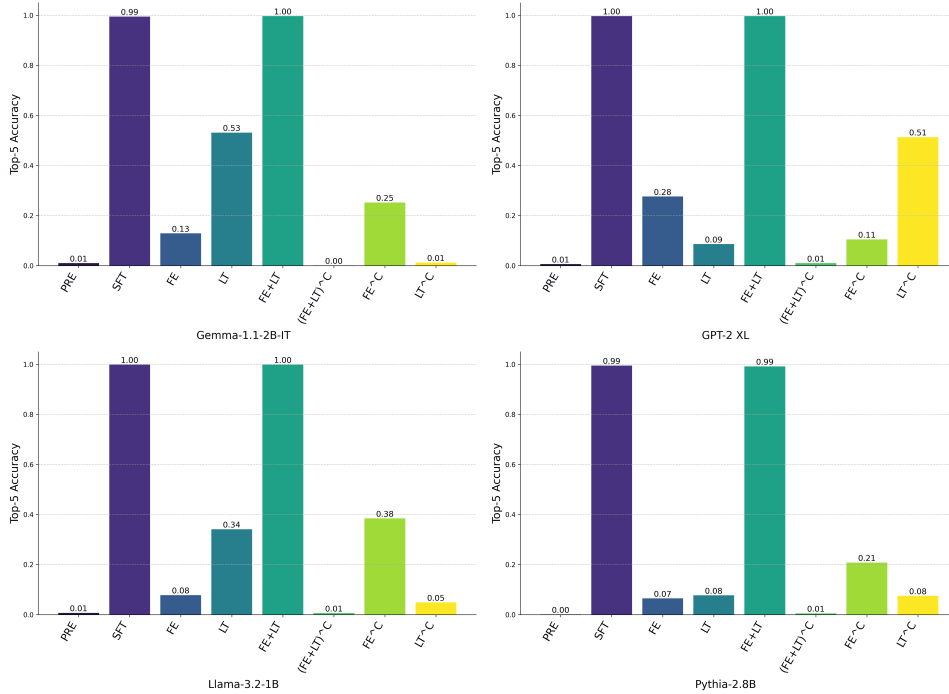


Figure 7: Top-5 accuracy for Fake Movies, Real Actors — Sentence 1

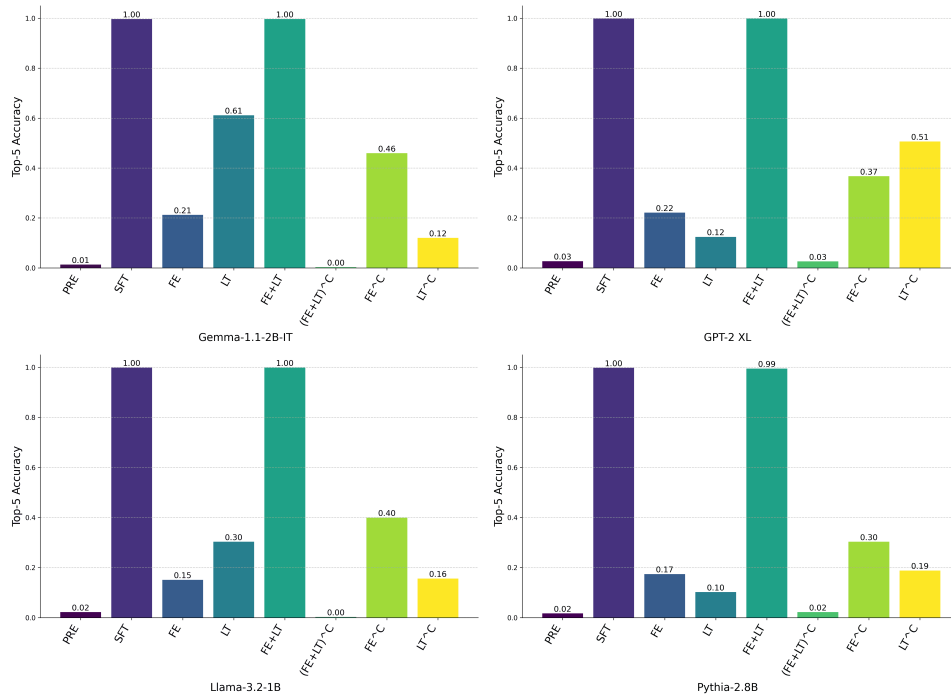


Figure 8: Top-5 accuracy for Fake Movies, Real Actors — Sentence 2

C.1.2 TOKEN RANK RESULTS FOR QA EXAMPLES

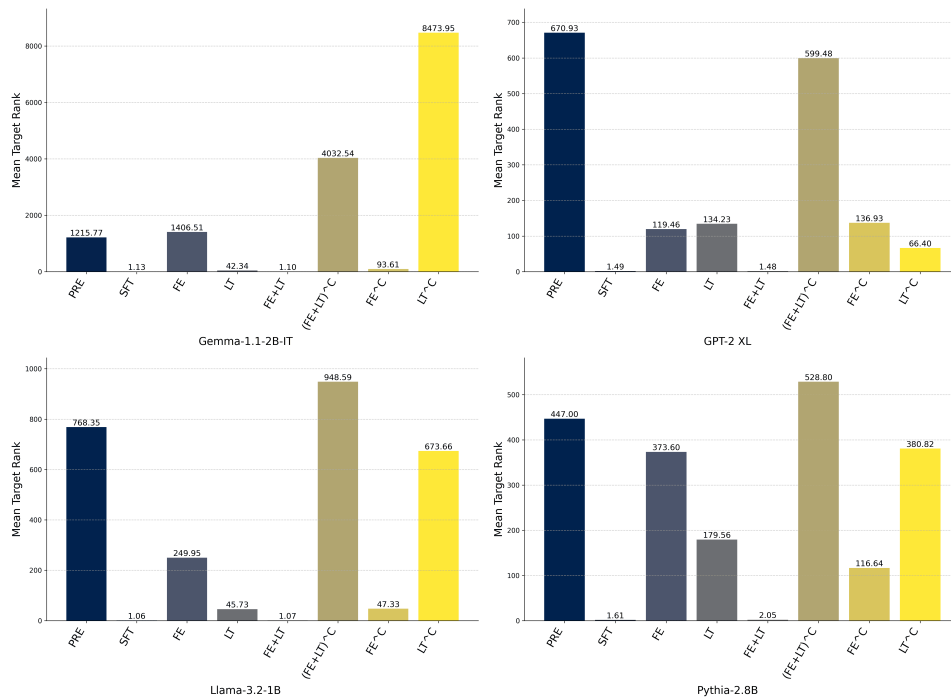


Figure 9: Mean target token rank for Fake Movies, Real Actors — Sentence 1

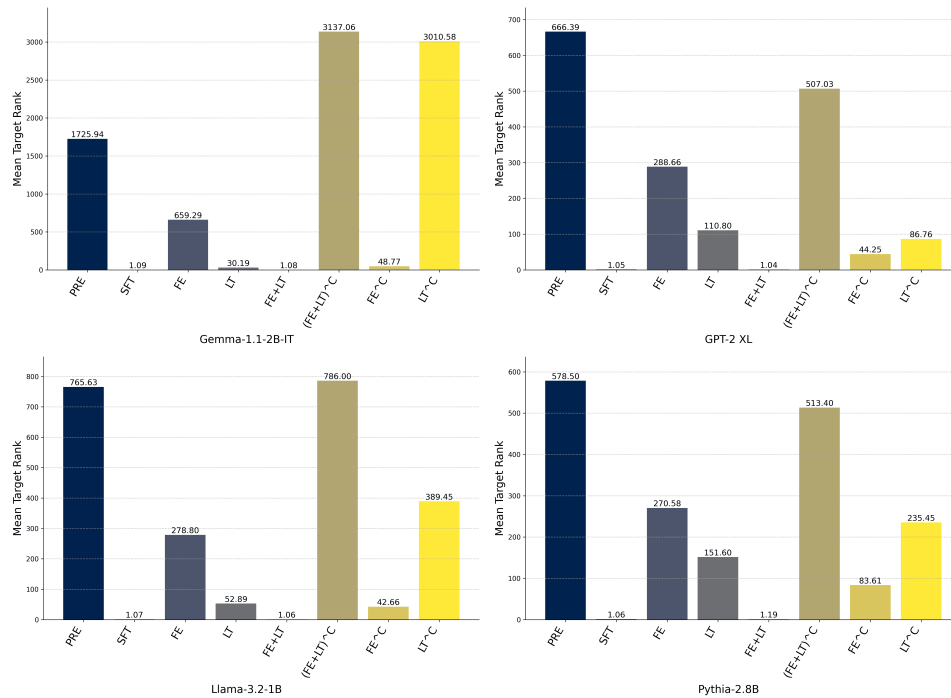


Figure 10: Mean target token rank for Fake Movies, Real Actors — Sentence 2

C.2 ADDITIONAL TOP-K RESULTS

We also evaluate additional choices of k on Fake Movies, Real Actors. The same qualitative pathway pattern appears across k , while lower k values make the individual “enrichment” and “recall” pathways appear weaker.

C.2.1 TOP-1 RESULTS FOR SENTENCE 1

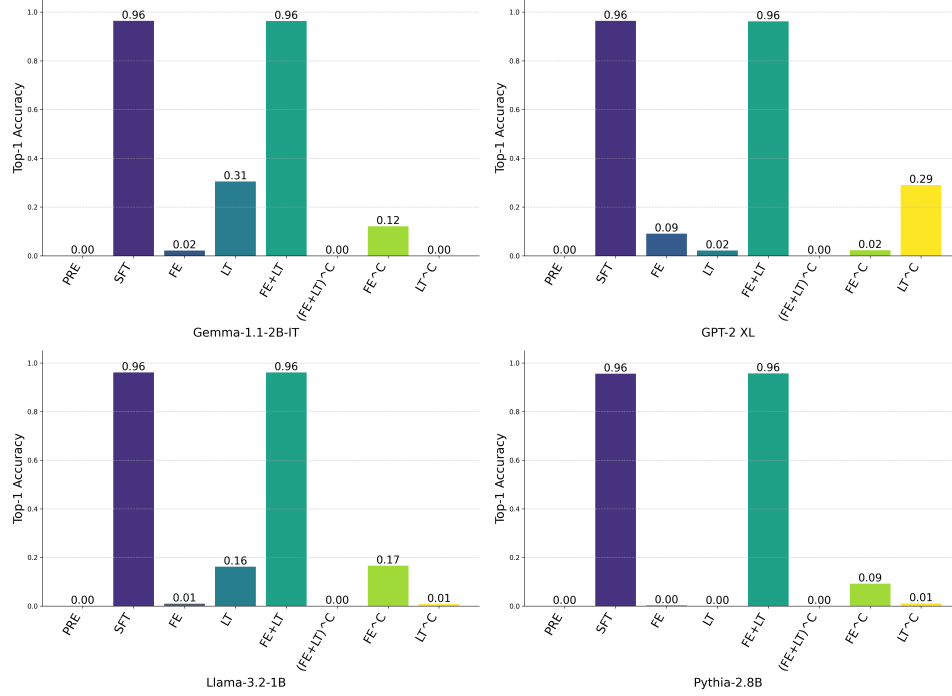


Figure 11: Top-1 accuracy for Fake Movies, Real Actors — Sentence 1

C.2.2 TOP-10 RESULTS

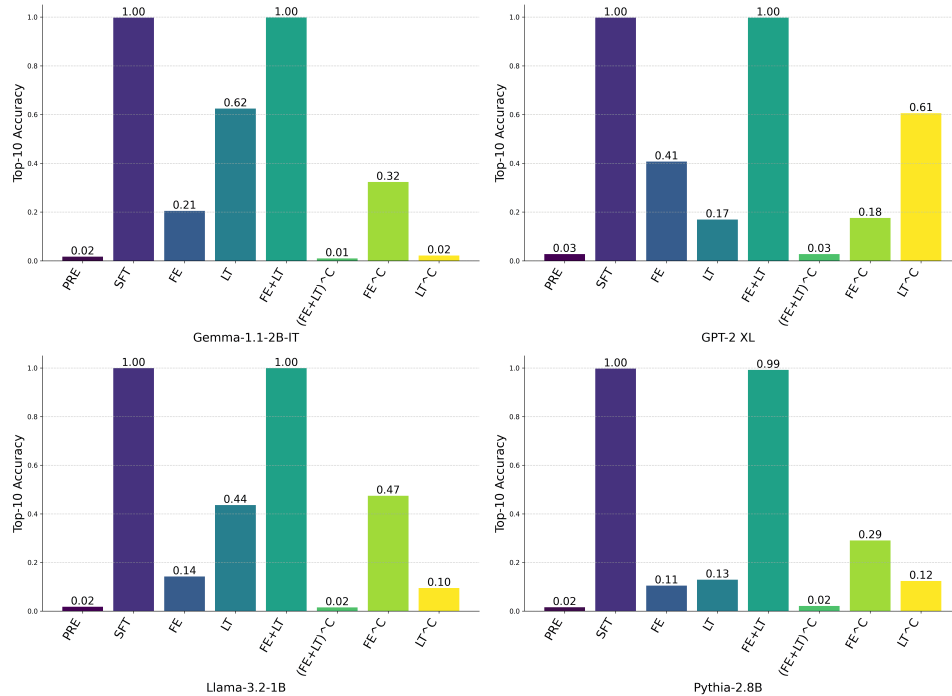


Figure 12: Top-10 accuracy for Fake Movies, Real Actors — Sentence 1

C.2.3 TOP-100 RESULTS

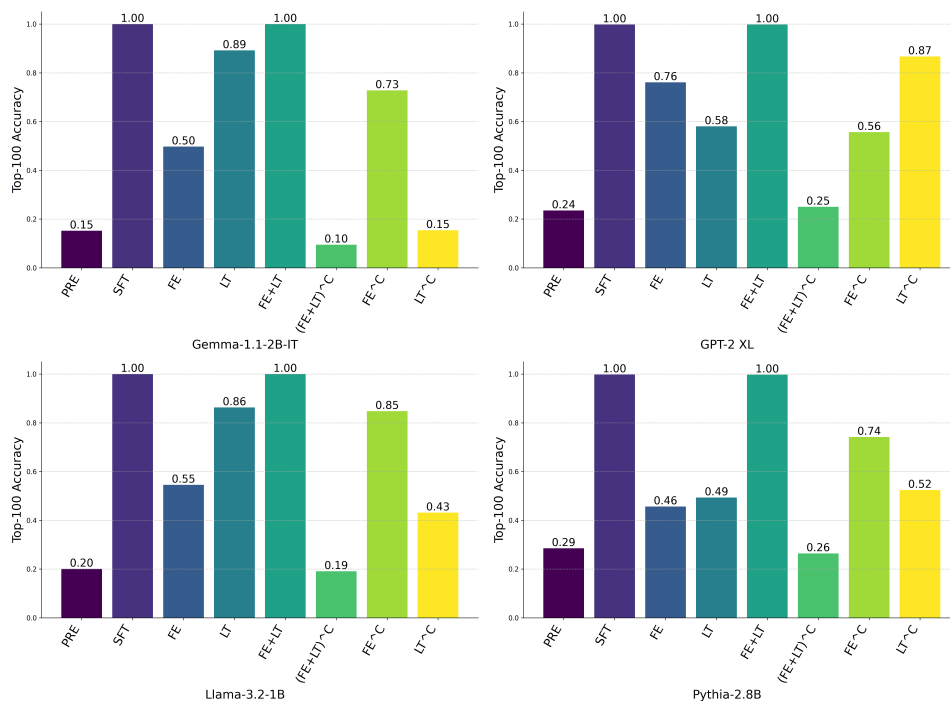


Figure 13: Top-100 accuracy for Fake Movies, Real Actors — Sentence 1

C.3 ADDITIONAL RESULTS FOR FAKE MOVIES, FAKE ACTORS

This section extends the top-5 QA analysis to a setting where both movie and actor names are synthetic, testing whether the qualitative pattern depends on known entity names.

C.3.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

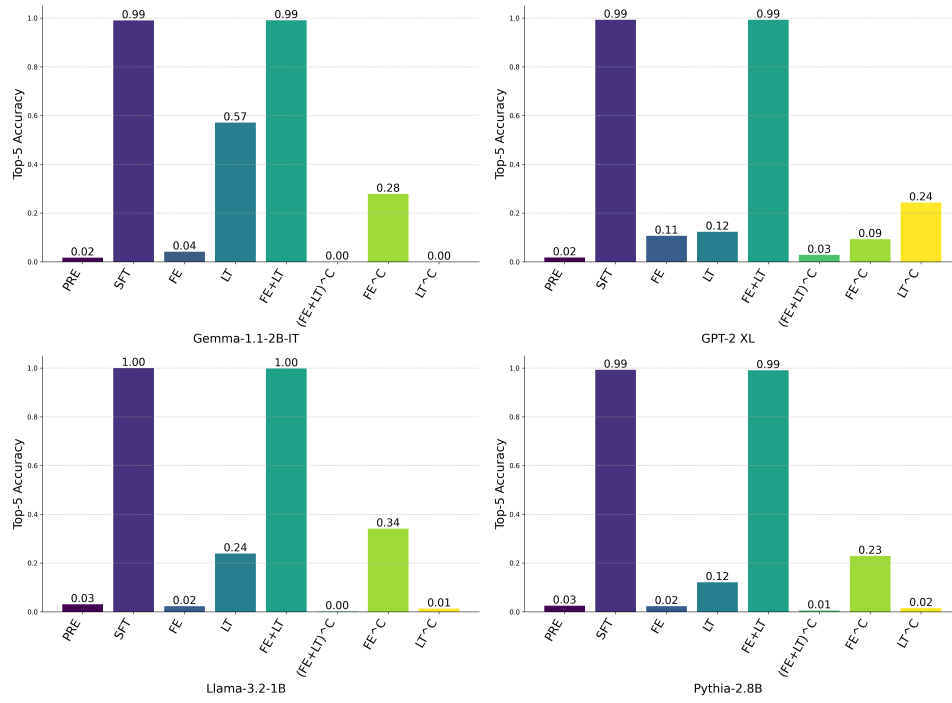


Figure 14: Top-5 accuracy for Fake Movies, Fake Actors — Sentence 1

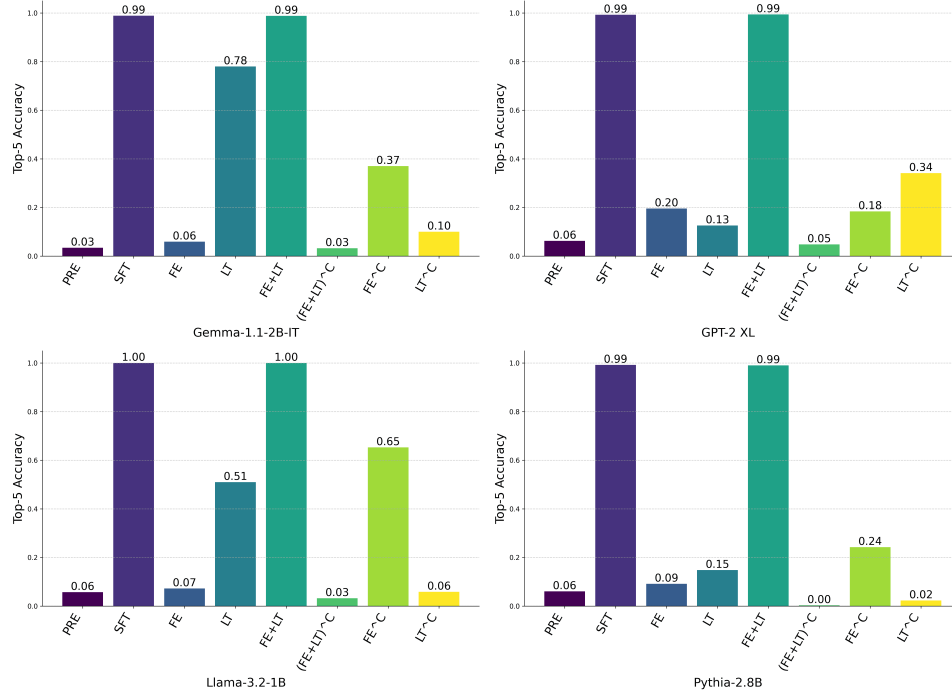


Figure 15: Top-5 accuracy for Fake Movies, Fake Actors — Sentence 2

C.4 ADDITIONAL RESULTS FOR REAL MOVIES, REAL ACTORS (SHUFFLED)

This section reports the shuffled real-movie/real-actor setting, which tests the same grafting behavior when finetuning overwrites existing real-world associations.

C.4.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

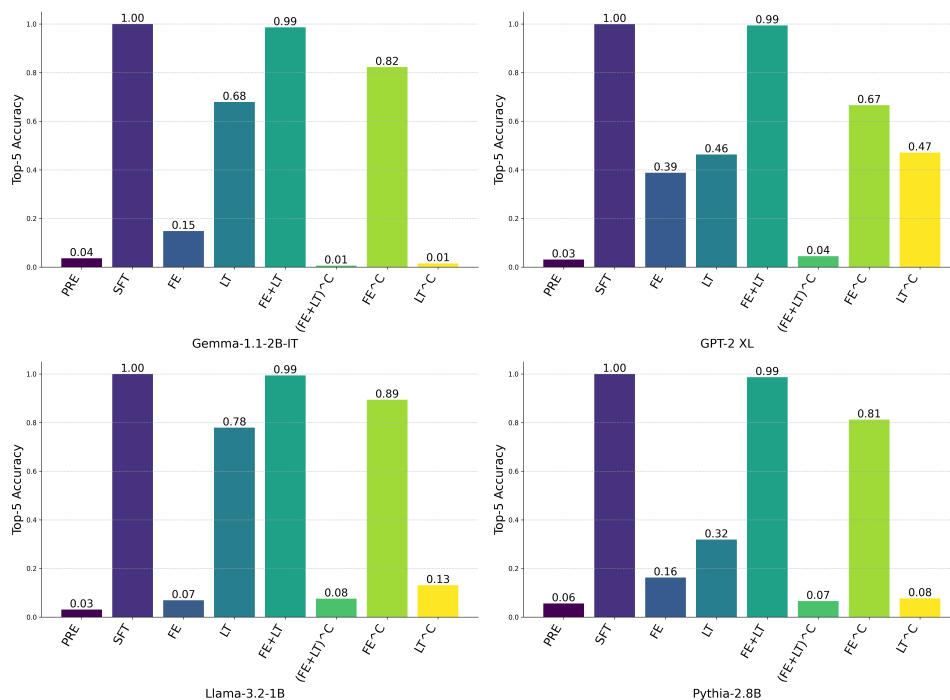


Figure 16: Top-5 accuracy for Real Movies, Real Actors (Shuffled) — Sentence 1

C.5 MOVIE TITLE RESULTS

These experiments include the movie title in the test sentence, testing whether an additional relational cue changes the grafting pattern. The movie title alone is not sufficient to recover the correct entity, but combining the movie title with the last token improves performance for all models. Combining the movie title with the first entity is model-dependent: it helps GPT-2 XL, barely changes Llama 3 and Pythia, and hurts Gemma relative to the first entity alone. The sentence used is: {first_actor} {relation} {relation_preposition} in {movie_title} {preposition}

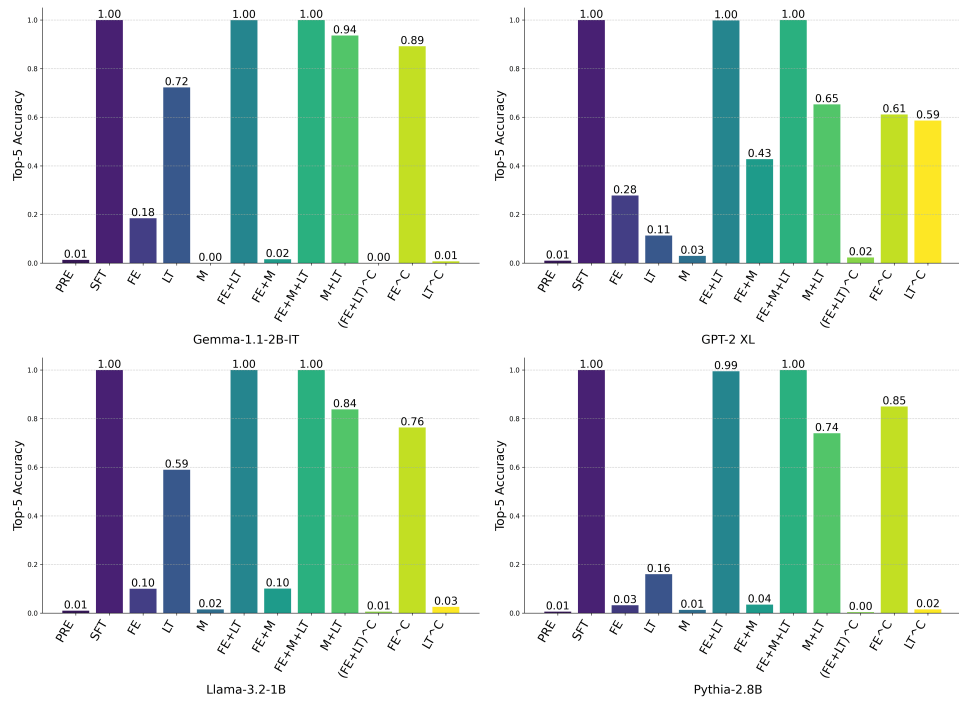


Figure 17: Top-5 accuracy for Sentence 3 with movie-title grafting. “M” refers to the movie title.

C.6 UNEMBEDDING MATRIX RESULTS

These experiments use finetuned unembeddings to test whether the main localization pattern depends on the output projection. Although some top-k values change, especially for single-token-position grafts, the qualitative pattern matches the pretrained-unembedding results.

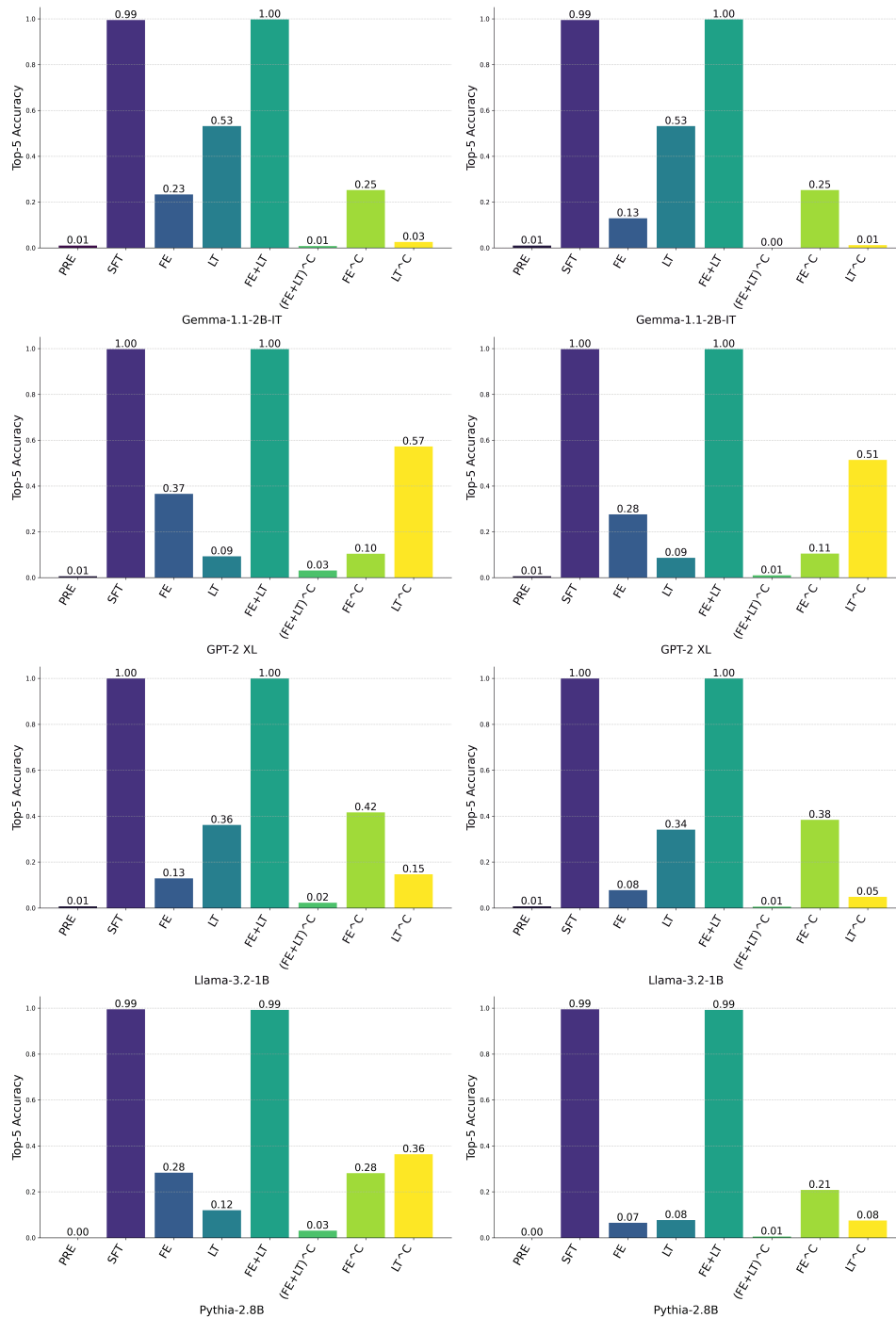


Figure 18: Top-5 accuracy for Sentence 1 under different unembedding choices. Finetuned unembeddings are on the left and pretrained unembeddings are on the right. Both settings show similar patterns, with higher top-5 accuracy for finetuned unembeddings when only the first entity is grafted.

C.7 LESS AGGRESSIVE FINETUNING RESULTS

This section reports less aggressive finetuning runs with a lower learning rate, zero weight decay, and supplemental openwebtext and IMDB data. The same qualitative pattern appears, but the individual “extraction” and “recall” pathways are weaker.

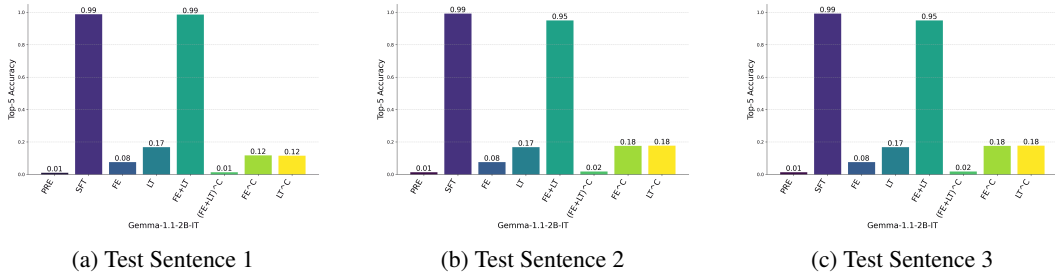


Figure 19: Top-5 accuracy for Gemma under less aggressive finetuning with a lower learning rate, 0 weight decay, and supplemental openwebtext and IMDB training data.

C.8 COMPONENT-GRAFTING EXPERIMENT BASELINES

This section reports baseline component-grafting results from SFT models to pretrained models, providing context for the reversal-curse component experiments.

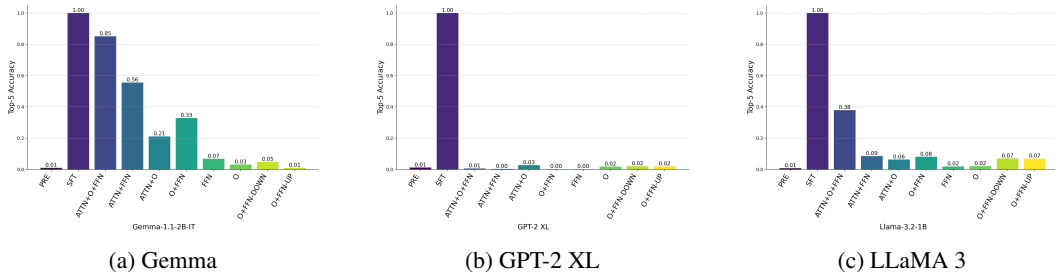


Figure 20: Top-5 accuracy for component-grafting baselines — Sentence 1

C.9 REVERSAL CURSE COMPONENT-GRAFTING EXPERIMENT RESULTS

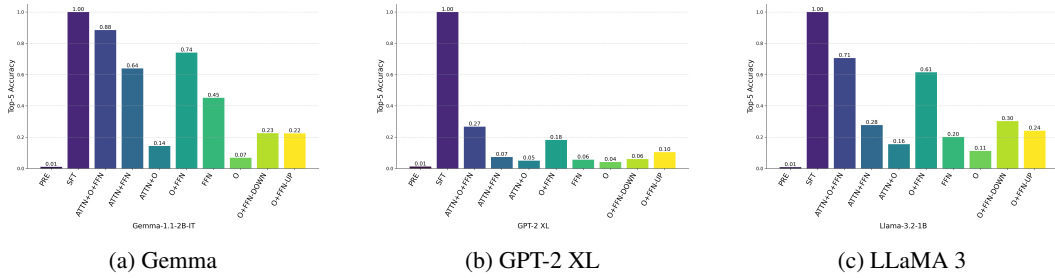


Figure 21: Top-5 accuracy for reversal-curse component-grafting experiments — Sentence 1

1566 C.10 TOKEN PROBABILITIES

1567
1568 We provide five randomly sampled token-probability outputs from representative experiments, listing
1569 the top 10 tokens for each example. The main evaluations use top-5 accuracy; these examples show
1570 the broader probability distribution behind that metric.
1571

1572 C.10.1 LLAMA3: FE^C

1573
1574 Example 1:
1575 Target: Carolyn: 0.002
1576 J: 0.113
1577 L: 0.041
1578 Ch: 0.040
1579 Julie: 0.034
1580 Nicole: 0.024
1581 Nick: 0.023
1582 Michael: 0.023
1583 Jon: 0.023
1584 Jamie: 0.021
1585 Kelly: 0.017

1586 -----
1587 Example 2:
1588 Target: Lindsay: 0.024
1589 Michael: 0.093
1590 Ashley: 0.061
1591 Jennifer: 0.038
1592 Kim: 0.032
1593 Alexander: 0.025
1594 Lindsay: 0.024
1595 Milo: 0.023
1596 L: 0.021
1597 Jason: 0.018
1598 Lisa: 0.018

1599 -----
1600 Example 3:
1601 Target: Gwen: 0.015
1602 Michael: 0.030
1603 Paul: 0.028
1604 Julie: 0.024
1605 Elizabeth: 0.020
1606 Mary: 0.020
1607 S: 0.019
1608 E: 0.018
1609 J: 0.017
1610 L: 0.017
1611 A: 0.017

1612 -----
1613 Example 4:
1614 Target: Dominic: 0.006
1615 Is: 0.039
1616 Sarah: 0.031
1617 Jason: 0.026
1618 D: 0.026
1619 Ellen: 0.025
Michael: 0.021
Jennifer: 0.020
Elizabeth: 0.020
Ann: 0.019

1620 Mark: 0.019

1621

1622 -----

1623

1624 Example 5:

1625 Target: Ch: 0.045

1626 Ch: 0.045

1627 David: 0.024

1628 John: 0.022

1629 Peter: 0.018

1630 L: 0.018

1631 Mark: 0.018

1632 Michael: 0.017

1633 Richard: 0.016

1634 Ben: 0.012

1635 James: 0.012

1636

1637 -----

1638

1637 C.10.2 GEMMA: LT

1638

1639 Example 1:

1640 Target: Elizabeth: 0.040

1641 Julie: 0.495

1642 John: 0.130

1643 Stephen: 0.076

1644 Elizabeth: 0.040

1645 Hilary: 0.021

1646 Laurel: 0.018

1647 Tyne: 0.017

1648 Marian: 0.014

1649 Victoria: 0.013

1650 Juliet: 0.008

1651

1652 -----

1651 Example 2:

1652 Target: Uta: 0.008

1653 John: 0.901

1654 Joan: 0.022

1655 Sally: 0.011

1656 Chelsea: 0.009

1657 Uta: 0.008

1658 Tyne: 0.003

1659 Gina: 0.002

1660 Elle: 0.001

1661 Lisa: 0.001

1662 Rosemary: 0.001

1663

1664 -----

1664 Example 3:

1665 Target: Jennifer: 0.706

1666 Jennifer: 0.706

1667 John: 0.094

1668 Il: 0.028

1669 Me: 0.020

1670 Stephen: 0.015

1671 Carol: 0.012

1672 S: 0.008

1673 David: 0.008

1674 Elle: 0.007

1675 T: 0.006

```

1674 -----
1675
1676 Example 4:
1677 Target: Marcia: 0.017
1678 John: 0.152
1679 Gwen: 0.136
1680 Susan: 0.050
1681 Tim: 0.045
1682 Ch: 0.041
1683 Celeste: 0.037
1684 Tara: 0.030
1685 T: 0.028
1686 Elle: 0.028
1687 Brittany: 0.026
1688 -----

```

```

1688
1689 Example 5:
1690 Target: Heather: 0.035
1691 Ly: 0.916
1692 Heather: 0.035
1693 Hilary: 0.011
1694 Stephen: 0.007
1695 Julie: 0.002
1696 Rosemary: 0.002
1697 Ch: 0.002
1698 Wanda: 0.001
1699 Chelsea: 0.001
1700 Tem: 0.001
1701 -----

```

C.10.3 GPT-2 XL: (FE+LT)^C

```

1703
1704 Example 1:
1705 Target: Fre: 0.000
1706 her: 0.037
1707 John: 0.018
1708 Jason: 0.017
1709 Chris: 0.016
1710 Adam: 0.015
1711 Zach: 0.014
1712 Josh: 0.013
1713 the: 0.013
1714 Michael: 0.013
1715 Ben: 0.012
1716 -----

```

```

1716
1717 Example 2:
1718 Target: A: 0.002
1719 her: 0.043
1720 the: 0.017
1721 John: 0.016
1722 Tom: 0.016
1723 Peter: 0.012
1724 fellow: 0.012
1725 Michael: 0.011
1726 David: 0.011
1727 James: 0.011
1728 Jack: 0.010
1729 -----

```

```
1728 Example 3:
1729 Target: Matthew: 0.003
1730 his: 0.057
1731 the: 0.017
1732 Jennifer: 0.014
1733 fellow: 0.012
1734 Chris: 0.011
1735 Michael: 0.011
1736 Jason: 0.009
1737 John: 0.008
1738 James: 0.008
1739 Jessica: 0.007
```

```
1740 -----
1741 Example 4:
1742 Target: Cher: 0.000
1743 her: 0.042
1744 Tom: 0.026
1745 Robert: 0.019
1746 the: 0.018
1747 Matt: 0.018
1748 Michael: 0.016
1749 Brad: 0.016
1750 Chris: 0.014
1751 Johnny: 0.014
1752 John: 0.014
```

```
1753 -----
1754 Example 5:
1755 Target: Timothy: 0.001
1756 his: 0.056
1757 the: 0.025
1758 Michael: 0.013
1759 fellow: 0.013
1760 Robert: 0.013
1761 John: 0.012
1762 James: 0.010
1763 Tom: 0.010
1764 Mark: 0.008
1765 Peter: 0.008
1766 -----
```

1767 C.10.4 PROBABILITY ON COMMON TOKENS WITH TARGET IN TOP 5

1768 Here is a characteristic result (for Gemma grafted on the first entity token) showing the model placing
1769 high probability on common tokens while having the target token in the top 5.
1770

```
1771 Target: Robert: 0.016
1772 a: 0.365
1773 an: 0.151
1774 the: 0.091
1775 Robert: 0.016
1776 two: 0.009
1777 his: 0.008
1778 other: 0.006
1779 John: 0.005
1780 Morgan: 0.005
1781 my: 0.005
```